

**NDAMULELO GODFREY TSHIVHULA**

**ST10488805**

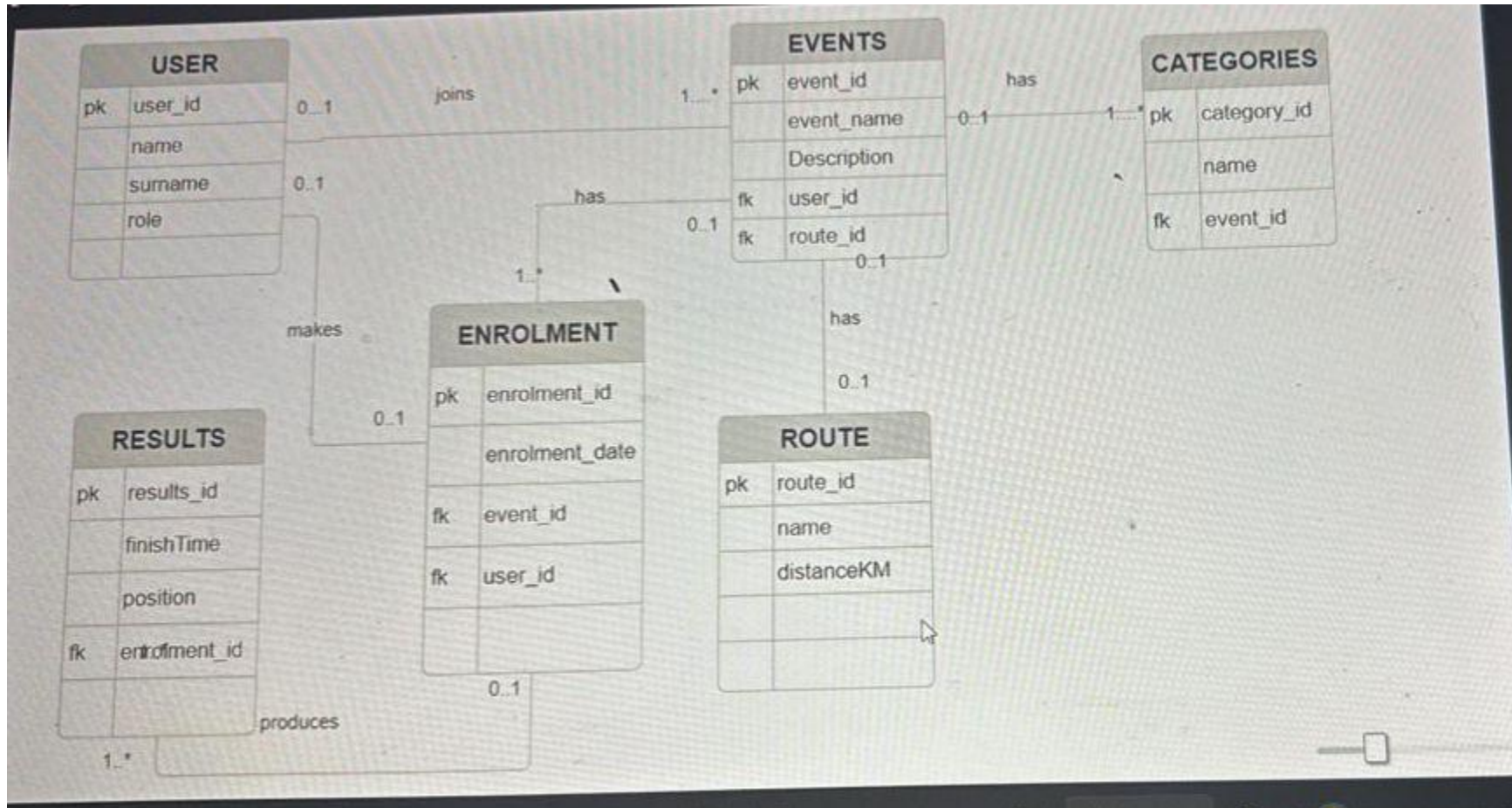
**PROGRAMMING 2B**

**GROUP 3**

**04 September 2026**

## Section A

### The ERD



## Section B

### Api endpoint

| HTTP Method | Route              | Description                                                         | Role Required                | Request Body                             | Expected Response                                                                                        |
|-------------|--------------------|---------------------------------------------------------------------|------------------------------|------------------------------------------|----------------------------------------------------------------------------------------------------------|
| POST        | /api/auth/register | Registers a new user account.                                       | None (public)                | { name, surname, email, password, role } | 201 Created – returns the new user object and an auth token.<br>409 Conflict – email already registered. |
| POST        | /api/auth/login    | Authenticates a user and returns an access token.                   | None (public)                | { email, password }                      | 200 OK – returns user details and an auth token.<br>401 Unauthorized – invalid email or password.        |
| GET         | /api/users/{id}    | Retrieves the profile details of a specific user.                   | Any (logged in)              | None                                     | 200 OK – returns the user object.<br>404 Not Found – user does not exist.                                |
| PUT         | /api/users/{id}    | Updates the profile details of a specific user.                     | Any (logged in, own profile) | { name, surname, role }                  | 200 OK – returns the updated user object.<br>404 Not Found – user does not exist.                        |
| GET         | /api/users         | Lists all registered users.                                         | Admin                        | None                                     | 200 OK – returns an array of user objects.                                                               |
| DELETE      | /api/users/{id}    | Deletes a specific user account.                                    | Admin                        | None                                     | 200 OK – confirmation message.<br>404 Not Found – user does not exist.                                   |
| GET         | /api/events        | Lists all events.                                                   | None (public)                | None                                     | 200 OK – returns an array of event objects.                                                              |
| GET         | /api/events/{id}   | Retrieves the details of a specific event.                          | None (public)                | None                                     | 200 OK – returns the event object.<br>404 Not Found – event does not exist.                              |
| POST        | /api/events        | Creates a new event, linked to the logged-in organiser and a route. | Organiser                    | { event_name, description, route_id }    | 201 Created – returns the new event object.<br>400 Bad Request – missing or invalid fields.              |

| HTTP Method | Route                       | Description                                         | Role Required             | Request Body                          | Expected Response                                                                           |
|-------------|-----------------------------|-----------------------------------------------------|---------------------------|---------------------------------------|---------------------------------------------------------------------------------------------|
| PUT         | /api/events/{id}            | Updates the details of a specific event.            | Organiser (owner) / Admin | { event_name, description, route_id } | 200 OK – returns the updated event object.<br>404 Not Found – event does not exist.         |
| DELETE      | /api/events/{id}            | Deletes a specific event.                           | Organiser (owner) / Admin | None                                  | 200 OK – confirmation message.<br>404 Not Found – event does not exist.                     |
| GET         | /api/routes                 | Lists all available routes.                         | None (public)             | None                                  | 200 OK – returns an array of route objects.                                                 |
| POST        | /api/routes                 | Creates a new route that events can be linked to.   | Organiser / Admin         | { name, distanceKM }                  | 201 Created – returns the new route object.<br>400 Bad Request – missing or invalid fields. |
| GET         | /api/events/{id}/categories | Lists all categories belonging to a specific event. | None (public)             | None                                  | 200 OK – returns an array of category objects.<br>404 Not Found – event does not exist.     |
| POST        | /api/events/{id}/categories | Adds a new category to a specific event.            | Organiser (owner) / Admin | { name }                              | 201 Created – returns the new category object.<br>404 Not Found – event does not exist.     |
| PUT         | /api/categories/{id}        | Updates the details of a specific category.         | Organiser (owner) / Admin | { name }                              | 200 OK – returns the updated category object.<br>404 Not Found – category does not exist.   |
| DELETE      | /api/categories/{id}        | Removes a specific category.                        | Organiser (owner) / Admin | None                                  | 200 OK – confirmation message.<br>404 Not Found – category does not exist.                  |
| GET         | /api/events/{id}/enrolments | Lists all enrolments for a specific event.          | Organiser (owner) / Admin | None                                  | 200 OK – returns an array of enrolment objects.<br>404 Not Found – event does not exist.    |
| POST        | /api/events/{id}/enrolments | Enrols the logged-in user into a specific event.    | Member (logged in)        | { enrolment_date }                    | 201 Created – returns the new enrolment record.                                             |

| HTTP Method | Route                        | Description                                      | Role Required       | Request Body              | Expected Response                                                                                                                    |
|-------------|------------------------------|--------------------------------------------------|---------------------|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
|             |                              |                                                  |                     |                           | 404 Not Found – event does not exist.<br>409 Conflict – user already enrolled.                                                       |
| GET         | /api/enrolments/{id}         | Retrieves the details of a specific enrolment.   | Any (owner) / Admin | None                      | 200 OK – returns the enrolment object.<br>404 Not Found – enrolment does not exist.                                                  |
| DELETE      | /api/enrolments/{id}         | Cancels/removes a specific enrolment.            | Any (owner) / Admin | None                      | 200 OK – confirmation message.<br>404 Not Found – enrolment does not exist.                                                          |
| GET         | /api/events/{id}/results     | Lists all results recorded for a specific event. | None (public)       | None                      | 200 OK – returns an array of result objects.<br>404 Not Found – event does not exist.                                                |
| POST        | /api/enrolments/{id}/results | Records a result for a specific enrolment.       | Organiser / Admin   | { finish_time, position } | 201 Created – returns the new result object.<br>404 Not Found – enrolment does not exist.<br>409 Conflict – result already recorded. |
| GET         | /api/results/{id}            | Retrieves the details of a specific result.      | None (public)       | None                      | 200 OK – returns the result object.<br>404 Not Found – result does not exist.                                                        |
| PUT         | /api/results/{id}            | Updates a specific result.                       | Organiser / Admin   | { finish_time, position } | 200 OK – returns the updated result object.<br>404 Not Found – result does not exist.                                                |
| DELETE      | /api/results/{id}            | Deletes a specific result.                       | Admin               | None                      | 200 OK – confirmation message.<br>404 Not Found – result does not exist.                                                             |

## Section C

### SQL code

```
CREATE DATABASE race_day;
```

```
use race_day;
```

```
CREATE TABLE [USER]
```

```
(  
    user_id INT IDENTITY(1,1) PRIMARY KEY,  
    name VARCHAR(50) NOT NULL,  
    surname VARCHAR(50) NOT NULL,  
    role VARCHAR(30) NOT NULL  
);
```

```
CREATE TABLE ROUTE
```

```
(  
    route_id INT IDENTITY(1,1) PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    distanceKM DECIMAL(6,2) NOT NULL,  
  
    CONSTRAINT CK_ROUTE_Distance  
        CHECK (distanceKM > 0)  
);
```

```
CREATE TABLE EVENTS
```

```
(  
    event_id INT IDENTITY(1,1) PRIMARY KEY,  
    event_name VARCHAR(100) NOT NULL,  
    Description VARCHAR(255),  
    user_id INT,  
    route_id INT,  
  
    CONSTRAINT FK_EVENTS_USER  
        FOREIGN KEY (user_id)  
        REFERENCES [USER](user_id),  
);
```

```
CONSTRAINT FK_EVENTS_ROUTE
    FOREIGN KEY (route_id)
    REFERENCES ROUTE(route_id)
);
```

```
CREATE TABLE CATEGORIES
(
    category_id INT IDENTITY(1,1) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    event_id INT NOT NULL,

    CONSTRAINT FK_CATEGORIES_EVENTS
        FOREIGN KEY (event_id)
        REFERENCES EVENTS(event_id)
);
```

```
CREATE TABLE ENROLMENT
(
    enrolment_id INT IDENTITY(1,1) PRIMARY KEY,
    enrolment_date DATE NOT NULL,
    event_id INT NOT NULL,
    user_id INT NOT NULL,

    CONSTRAINT FK_ENROLMENT_EVENTS
        FOREIGN KEY (event_id)
        REFERENCES EVENTS(event_id),

    CONSTRAINT FK_ENROLMENT_USER
        FOREIGN KEY (user_id)
        REFERENCES [USER](user_id)
);
```

```
CREATE TABLE RESULTS
(
    results_id INT IDENTITY(1,1) PRIMARY KEY,
    finishTime TIME NOT NULL,
    position INT NOT NULL,
    enrolment_id INT NOT NULL,
```

**CONSTRAINT FK\_RESULTS\_ENROLMENT**  
**FOREIGN KEY (enrolment\_id)**  
**REFERENCES ENROLMENT(enrolment\_id),**

**CONSTRAINT CK\_RESULTS\_Position**  
**CHECK (position > 0)**

**);**

**INSERT INTO [USER]**  
**(name, surname, role)**  
**VALUES**  
**('Mpho', 'Nkosi', 'Organiser'),**  
**('Lerato', 'Molefe', 'Organiser'),**  
**('Kabelo', 'Dlamini', 'Participant'),**  
**('Nokuthula', 'Maseko', 'Participant');**

**INSERT INTO ROUTE**  
**(name, distanceKM)**  
**VALUES**  
**('Polokwane City Route', 10.00),**  
**('Tzaneen Community Route', 5.00),**  
**('Makhado Cycling Route', 25.00);**

**INSERT INTO EVENTS**  
**(event\_name, Description, user\_id, route\_id)**  
**VALUES**  
**('Limpopo Fun Run',**  
**'Annual community running event', 1, 1),**

**('Charity Walk',**  
**'Community fundraising walk', 2,2),**

**('Northern Cycle Challenge',**  
**'Cycling competition', 1, 3);**

**INSERT INTO CATEGORIES**  
**(name, event\_id)**



**VALUES**

**('10KM Open', 1),  
('5KM Fun Run', 1),  
('5KM Family Walk', 2),  
('10KM Walk', 2),  
('25KM Cycle', 3);**

**INSERT INTO ENROLMENT**

**(enrolment\_date, event\_id, user\_id)**

**VALUES**

**('2026-09-01', 1, 3),  
('2026-09-02', 1, 4),  
('2026-09-03', 2, 3),  
('2026-09-04', 3, 4);**

**INSERT INTO RESULTS**

**(finishTime, position, enrolment\_id)**

**VALUES**

**('00:45:32', 1, 1),  
('00:48:15', 2, 2);**

**SELECT \* FROM [USER];**

**SELECT \* FROM EVENTS;**

**SELECT \* FROM CATEGORIES;**

**SELECT \* FROM ROUTE;**

**SELECT \* FROM ENROLMENT;**

**SELECT \* FROM RESULTS;**

100 % ✓ No issues found Ln: 139, Ch: 1 (22 chars, 2 lines) SPC LF Windows 1252

Results Messages

|   | user_id | name      | surname | role        |
|---|---------|-----------|---------|-------------|
| 1 | 1       | Mpho      | Nkosi   | Organiser   |
| 2 | 2       | Lerato    | Molefe  | Organiser   |
| 3 | 3       | Kabelo    | Dlamini | Participant |
| 4 | 4       | Nokuthula | Maseko  | Participant |

✓ Query executed successfully. (localdb)\MSSQLLocalDB (15.... LabVM2049939\Student (65) race\_day 00:00:00 Row: 1, Col: 1 4 rows

100 % ✓ No issues found Ln: 142, Ch: 1 (21 chars, 1 lines) SPC LF Windows 1252

Results Messages

|   | event_id | event_name               | Description                    | user_id | route_id |
|---|----------|--------------------------|--------------------------------|---------|----------|
| 1 | 1        | Limpopo Fun Run          | Annual community running event | 1       | 1        |
| 2 | 2        | Charity Walk             | Community fundraising walk     | 2       | 2        |
| 3 | 3        | Northern Cycle Challenge | Cycling competition            | 1       | 3        |

✓ Query executed successfully. (localdb)\MSSQLLocalDB (15.... LabVM2049939\Student (65) race\_day 00:00:00 Row: 1, Col: 1 3 rows

100 % ✓ No issues found Ln: 143, Ch: 1 (26 chars, 2 lines) SPC LF Windows 1252

Results Messages

|   | category_id | name            | event_id |
|---|-------------|-----------------|----------|
| 1 | 1           | 10KM Open       | 1        |
| 2 | 2           | 5KM Fun Run     | 1        |
| 3 | 3           | 5KM Family Walk | 2        |
| 4 | 4           | 10KM Walk       | 2        |
| 5 | 5           | 25KM Cycle      | 3        |

✓ Query executed successfully. (localdb)\MSSQLLocalDB (15.... LabVM2049939\Student (65) race\_day 00:00:00 Row: 1, Col: 1 5 rows

100 % ✓ No issues found Ln: 146, Ch: 1 (20 chars, 1 lines) SPC LF Windows 1252

Results Messages

|   | route_id | name                    | distanceKM |
|---|----------|-------------------------|------------|
| 1 | 1        | Polokwane City Route    | 10.00      |
| 2 | 2        | Tzaneen Community Route | 5.00       |
| 3 | 3        | Makhado Cycling Route   | 25.00      |

✓ Query executed successfully. (localdb)\MSSQLLocalDB (15.... LabVM2049939\Student (65) race\_day 00:00:00 Row: 1, Col: 1 3 rows

100 % No issues found Ln: 148, Ch: 1 (24 chars, 1 lines) SPC LF Windows 1252

Results Messages

|   | enrolment_id | enrolment_date | event_id | user_id |
|---|--------------|----------------|----------|---------|
| 1 | 1            | 2026-09-01     | 1        | 3       |
| 2 | 2            | 2026-09-02     | 1        | 4       |
| 3 | 3            | 2026-09-03     | 2        | 3       |
| 4 | 4            | 2026-09-04     | 3        | 4       |

Query executed successfully. (localdb)\MSSQLLocalDB (15.... LabVM2049939\Student (65) race\_day 00:00:00 Row: 1, Col: 1 4 rows

100 % No issues found Ln: 150, Ch: 1 (22 chars, 1 lines) SPC LF Windows 1252

Results Messages

|   | results_id | finishTime       | position | enrolment_id |
|---|------------|------------------|----------|--------------|
| 1 | 1          | 00:45:32.0000000 | 1        | 1            |
| 2 | 2          | 00:48:15.0000000 | 2        | 2            |

Query executed successfully. (localdb)\MSSQLLocalDB (15.... LabVM2049939\Student (65) race\_day 00:00:00 Row: 1, Col: 1 2 rows

## References

- Microsoft (2024) CREATE TABLE (Transact-SQL). Microsoft Learn. Available at: Microsoft Learn – CREATE TABLE (Transact-SQL) (Accessed: 3 September 2026).
- Microsoft (2024) Primary and foreign key constraints. Microsoft Learn. Available at: Microsoft Learn – Primary and foreign key constraints (Accessed: 3 September 2026).
- Coronel, C. and Morris, S. (2019) Database Systems: Design, Implementation, & Management. 13th edn. Boston: Cengage Learning.